Lab Goal : This lab was designed to teach you more about Binary Trees.

Lab Description : Write a binary search tree class. You must write the following methods ::

inOrder()	this method will print the tree using a LDR traversal
preOrder()	this method will print the tree using a DLR traversal
postOrder()	this method will print the tree using a LRD traversal
revOrder()	this method will print the tree using a RDL traversal
toString()	returns a String that lists the values in their natural order
clear()	removes all nodes from the tree
search(value)	write a method to **search the tree for a value** and return true or false
getNumNodes()	returns the number of nodes in the tree
getNumLeaves()	returns the number of leaves in the tree
getNumLevels()	returns the number of levels in the tree
getHeight()	returns the height of the tree
getLargest()	returns the largest value
getSmallest()	returns the smallest value
getDiameter()	returns the maximum diameter of the tree
isPerfect()	a boolean method that indicates whether this tree is perfect (i.e. every level is full)
isFull()	indicates whether this tree is full (i.e. every node has 0 or 2 children)
remove(value)	write a method to **remove** a node from the tree – must be recursive

---

Stage 1

```
tree.add(90)
tree.add(90) <-- shouldn't add!!!
tree.add(100)
tree.add(80)
tree.add(70)
tree.add(98)
```

*** Expected tree structure after these additions ***

```
          90
      80      100
   70      98
```

Level order traversal with nulls in place of missing nodes:
90 | 80 100 | 70 null 98

```
IN ORDER from toString: 70 80 90 98 100
IN ORDER: 70 80 90 98 100
PRE ORDER: 90 80 70 100 98
POST ORDER: 70 80 98 100 90
REVERSE ORDER: 100 98 90 80 70

The tree is not full.
The tree is not perfect.
The tree contains 100!
The does not contain 114!

Number of nodes is 1
Number of leaves is 2
Number of levels is 3
Tree height is 2
The smallest tree node 70
The largest tree node 100
Tree diameter is 5
Sum of all nodes: 438.0
```

Stage 2

```
tree.add(85)
```

*** Expected tree structure after these additions ***

```
          90
      80      100
   70   85   98
```

Level order traversal with nulls in place of missing nodes:
90 | 80 100 | 70 85 98

```
IN ORDER from toString: 70 80 85 90 98 100
IN ORDER: 70 80 85 90 98 100
PRE ORDER: 90 80 70 85 100 98
POST ORDER: 70 85 80 98 100 90
REVERSE ORDER: 100 98 90 85 80 70

The tree is not full.
The tree is not perfect.
The tree contains 100!
The does not contain 114!

Number of nodes is 1
Number of leaves is 3
Number of levels is 3
Tree height is 2
The smallest tree node 70
The largest tree node 100
Tree diameter is 5
Sum of all nodes: 523.0
```

Stage 3

```
tree.add(98)
tree.add(120)

*** Expected tree structure after these additions ***

                90
            80          100
        70      85      98      120

Level order traversal with nulls in place of missing nodes:
90 | 80 100 | 70 85 98 120 |

IN ORDER from toString: 70 80 85 90 98 100 120
          IN ORDER: 70 80 85 90 98 100 120
        PRE ORDER: 90 80 70 85 100 98 120
       POST ORDER: 70 85 80 98 120 100 90
    REVERSE ORDER: 120 100 98 90 85 80 70
```

```
The tree is full.
The tree is not perfect.
The tree contains 100!
The does not contain 114!

Number of nodes is 1
Number of leaves is 4
Number of levels is 3
Tree height is 2
The smallest tree node 70
The largest tree node 120
Tree diameter is 5
Sum of all nodes: 643.0
```